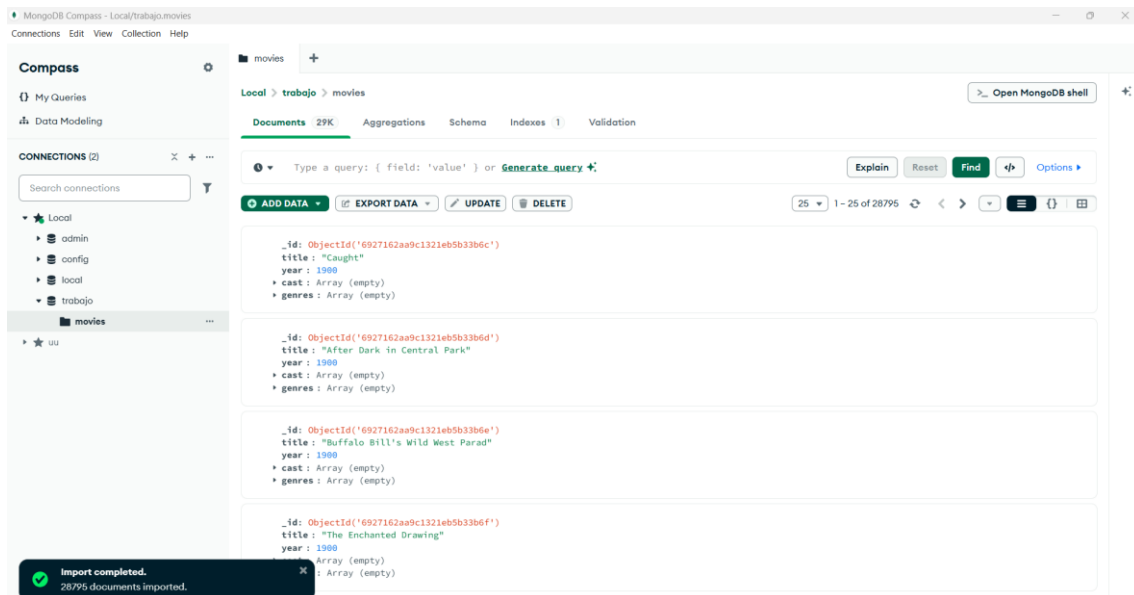
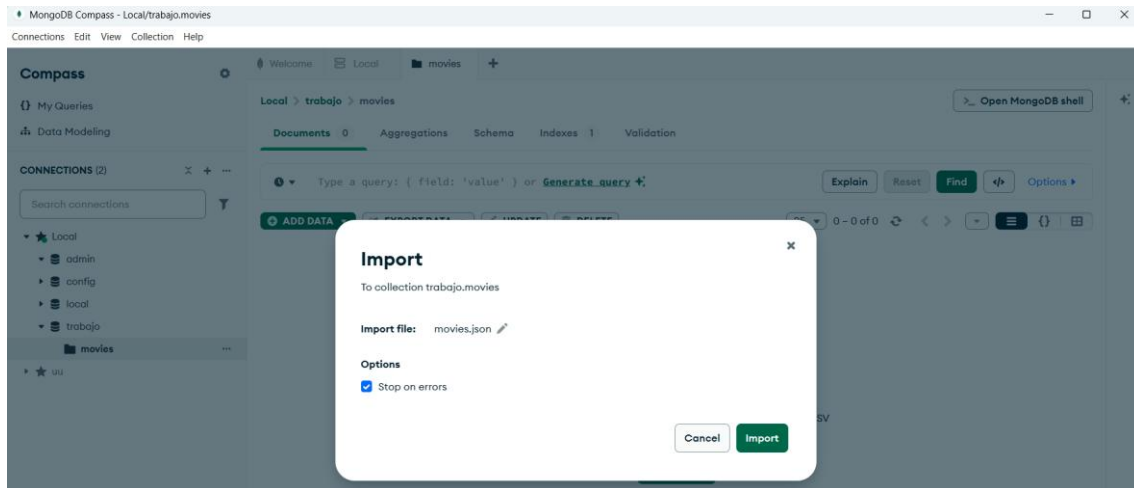




‘Base de Datos NoSQL’

Pablo Blas Hernández

0. Realizar la importación del json en una colección llamada “movies”.



Importa un total de 28795 documentos (películas).

1. Analizar con find la colección.

`db.movies.find()`

Key	Value	Type
(1) 692710eeade0aa4ff62887a3	{ title : "Caught", year : 1900, cast : [], genres : [] } (5 fields)	Document
_id (asc index)	692710eeade0aa4ff62887a3	ObjectId
title	Caught	String
year	1900	Int32
cast	Array[0]	Array
genres	Array[0]	Array
(2) 692710eeade0aa4ff62887a4	{ title : "After Dark in Central Park", year : 1900, cast : [], genres : [] } (5 fields)	Document
_id (asc index)	692710eeade0aa4ff62887a4	ObjectId
title	After Dark in Central Park	String
year	1900	Int32
cast	Array[0]	Array
genres	Array[0]	Array
(3) 692710eeade0aa4ff62887a5	{ title : "Buffalo Bill's Wild West Parad", year : 1900, cast : [], genres : [] } (5 fields)	Document
(4) 692710eeade0aa4ff62887a6	{ title : "The Enchanted Drawing", year : 1900, cast : [], genres : [] } (5 fields)	Document
(5) 692710eeade0aa4ff62887a7	{ title : "Clowns Spinning Hats", year : 1900, cast : [], genres : [] } (5 fields)	Document
(6) 692710eeade0aa4ff62887a8	{ title : "Capture of Boer Battery by British", year : 1900, cast : [], genres : ["Short"] } (5 fields)	Document
(7) 692710eeade0aa4ff62887a9	{ title : "Feeding Sea Lions", year : 1900, cast : ["Paul Boyton"], genres : [] } (5 fields)	Document
(8) 692710eeade0aa4ff62887aa	{ title : "How to Make a Fat Wife Out of Two Lean Ones", year : 1900, cast : [], genres : [] } (5 fields)	Document
(9) 692710eeade0aa4ff62887ab	{ title : "Searching Ruins on Broadway, Galveston, for Dead Bodies", year : 1900, cast : [], genres : [] } (5 fields)	Document
(10) 692710eeade0aa4ff62887ac	{ title : "The Tribulations of an Amateur Photographer", year : 1900, cast : [], genres : [] } (5 fields)	Document
(11) 692710eeade0aa4ff62887ad	{ title : "Trouble in Hogan's Alley", year : 1900, cast : [], genres : ["Comedy"] } (5 fields)	Document

Mostramos una salida de ejemplo con un documento aleatorio de la base de datos (la primera película por ejemplo), para sí mostrar los campos de cada documento:

`db.movies.findOne()`

Key	Value	Type
(1) 692710eeade0aa4ff62887a3	{ title : "Caught", year : 1900, cast : [], genres : [] } (5 fields)	Document
_id	692710eeade0aa4ff62887a3	ObjectId
title	Caught	String
year	1900	Int32
cast	Array[0]	Array
genres	Array[0]	Array

```
{
  "_id" : ObjectId("692710eeade0aa4ff62887a3"),
  "title" : "Caught",
  "year" : 1900,
  "cast" : [ ],
  "genres" : [ ]
}
```

2. Contar cuántos documentos (películas) tiene cargado.

`db.movies.countDocuments()`



Tiene un total de 28795 documentos (coincide con el nº de documentos importado).

3. Insertar una película.

```
db.movies.insertOne(
```

```
{
  "title": "EL Misterio del Código Omega",
  "year": 2025,
  "cast": ["Ana Fernández", "Javier Rey", "Elena Anaya"],
  "genres": ["Thriller", "Ciencia Ficción", "Misterio"]
}
);
```



Buscamos la película que hemos insertado, para confirmar que se ha registrado:

```
db.movies.find(
```

```
{
  "title": "EL Misterio del Código Omega"
}
);
```

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible with 'trabajo' selected, containing 'movies' and 'users' collections. The 'movies' collection is expanded, showing a document with the title 'EL Misterio del Código Omega'. The main panel displays the query `db.movies.find({ 'title': 'EL Misterio del Código Omega' })` and the resulting document in a table view. The document contains fields for `_id`, `title`, `year`, `cast`, and `genres`.

Key	Value	Type
(1) 692712f22885762ab8e8e8c4	{ title: "EL Misterio del Código Omega", year: 2025 } (5 fields)	Document
_id (asc index)	692712f22885762ab8e8e8c4	ObjectId
title	EL Misterio del Código Omega	String
year	2025	Int32
cast	Array[3]	Array
0	Ana Fernández	String
1	Javier Rey	String
2	Elena Anaya	String
genres	Array[3]	Array
0	Thriller	String
1	Ciencia Ficción	String
2	Misterio	String

Hacemos la búsqueda por el título de la película, y efectivamente, encontramos el registro con los datos de la película que hemos insertado. Mucho ojo, porque si hubiésemos insertado una película con un título ya existente en la base de datos, habría dos registros con ese título, y al usar ‘find’ por el título de la película, obtendríamos ambos documentos, y usando ‘findOne’, podríamos obtener la película que ya estaba originalmente en la base de datos y no la insertada por nosotros.

4. Borrar la película insertada en el punto anterior (en el 3).

```
db.movies.deleteOne(
```

```
{
  "title": "EL Misterio del Código Omega"
})
```

The screenshot shows the MongoDB Compass interface after executing the `deleteOne` query. The main panel displays the query `db.movies.deleteOne({ 'title': 'EL Misterio del Código Omega' })` and the resulting document in a table view. The document contains fields for `acknowledged` and `deletedCount`.

Key	Value	Type
(1)	{ acknowledged: true, deletedCount: 1 }	Object
acknowledged	true	Bool
deletedCount	1	Int32

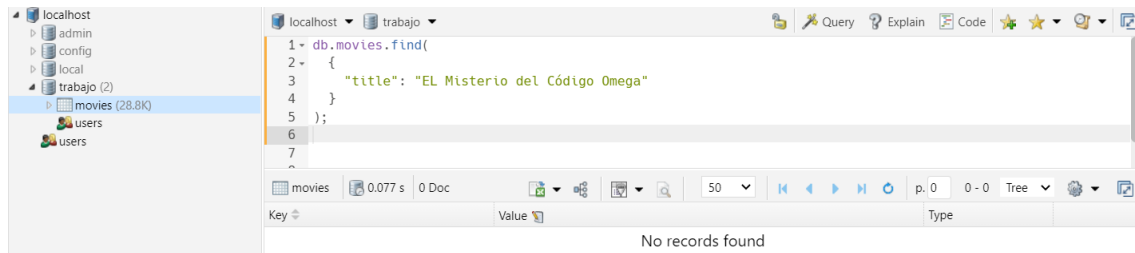
Buscamos el título de la película que acabamos de eliminar. El resultado debe ser que no encuentra ese registro:

```
db.movies.find(
```

```
{
  "title": "EL Misterio del Código Omega"
})
```

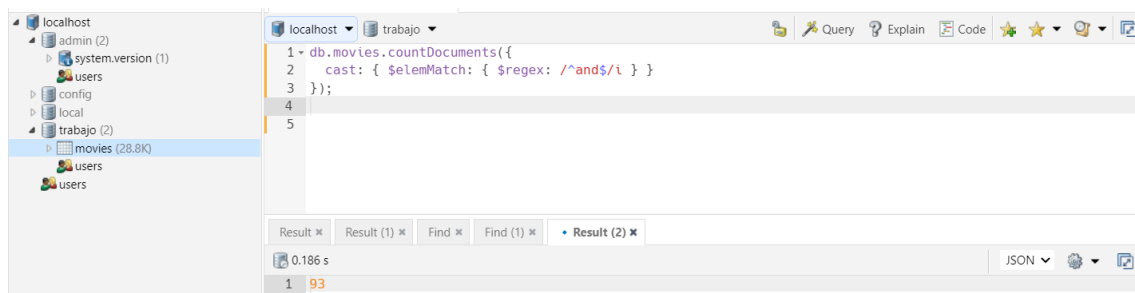
```
}
```

```
);
```



5. Contar cuantas películas tienen actores (cast) que se llaman “and”. Estos nombres de actores están por ERROR.

```
db.movies.countDocuments({
  cast: { $elemMatch: { $regex: /^and$/i } }
});
```



Hay un total de 93 películas con el error de identificar “and” como si fuese el nombre de un actor. Mostramos algunos ejemplos de cómo se ven estos errores. Observamos que guarda el “and” como si fuese un actor más:

```
db.movies.find(
  { cast: { $elemMatch: { $regex: /^and$/i } } },
  { _id: 0, title: 1, cast: 1 }
);
```

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible: localhost > trabajo > movies (28.8K). The main panel displays a query result for the 'movies' collection. The query is: `1 - db.movies.find( 2 { cast: { $elemMatch: { $regex: /^and$/i } } }, 3 { _id: 0, title: 1, cast: 1 } 4 );`. The result shows 93 documents. The first document (index 1) is expanded, showing its 'cast' array: `[ "Betty Nansen", "and", "Edward José" ]`. The value "and" is circled in red, indicating it is the target of the update operation.

6. Actualizar los documentos cuyo actor (cast) tenga por error el valor “and” como si realmente fuera un actor. Para ello, se debe sacar únicamente ese valor del array cast. Por lo tanto, no se debe eliminar ni el documento (película) ni su array cast con el resto de actores.

`db.movies.updateMany(`

`{ cast: { $regex: /^and$/i } }, // Busca el elemento exacto "and" en el array 'cast'`

`{ $pull: { cast: { $regex: /^and$/i } } } // Elimina ese elemento del array 'cast'`

`);`

The screenshot shows the MongoDB Compass interface after running the updateMany operation. The query is: `1 - db.movies.updateMany( 2 { cast: { $regex: /^and$/i } }, // busca los que tienen exactamente "and" 3 { $pull: { cast: { $regex: /^and$/i } } } // saca "and" del array 4 );`. The result shows 1 document (index 1) with the following fields: `{ acknowledged: true, matchedCount: 93, modifiedCount: 93 }`. The 'matchedCount' and 'modifiedCount' are both 93, indicating that all 93 documents were successfully updated.

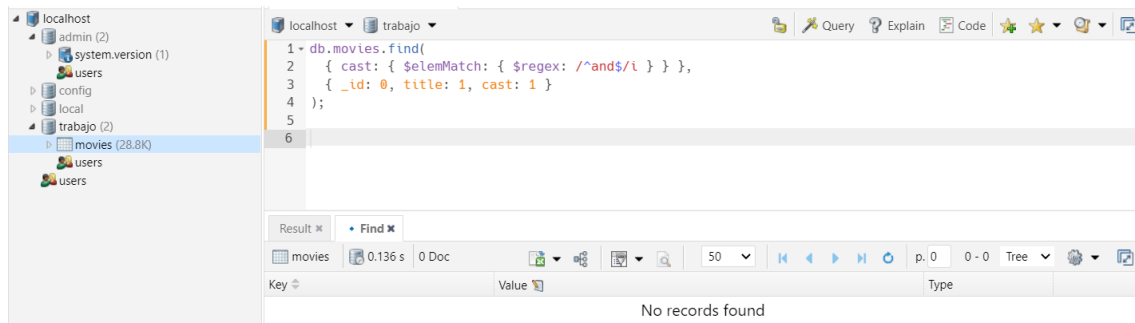
Comprobamos que efectivamente “and” ya no se guarda como si fuese un actor más:

`db.movies.find(`

`{ cast: { $elemMatch: { $regex: /^and$/i } } },`

`{ _id: 0, title: 1, cast: 1 }`

`);`



7. Contar cuantos documentos (películas) tienen el array 'cast' vacío.

```
db.movies.countDocuments({
  cast: { $size: 0 }
});
```



Un total de 986 documentos tienen el array 'cast' vacío. Mostramos ejemplos:

```
db.movies.find(
  { cast: { $size: 0 } },
  { _id: 0, title: 1, year: 1, cast: 1 }
);
```



```

1 db.movies.find(
2   { cast: { $size: 0 } },
3   { _id: 0, title: 1, year: 1, cast: 1 }
4 );

```

Key	Value	Type
(1)	{ title: "Caught", year: 1900, cast: [] }	Object
title	Caught	String
year	1900	Int32
cast	Array[]	Array
(2)	{ title: "After Dark in Central Park", year: 1900, cast: [] }	Object
title	After Dark in Central Park	String
year	1900	Int32
cast	Array[]	Array
(3)	{ title: "Buffalo Bill's Wild West Parad", year: 1900, cast: [] }	Object
(4)	{ title: "The Enchanted Drawing", year: 1900, cast: [] }	Object
(5)	{ title: "Clowns Spinning Hats", year: 1900, cast: [] }	Object
(6)	{ title: "Capture of Boer Battery by British", year: 1900, cast: [] }	Object
(7)	{ title: "How to Make a Fat Wife Out of Two Lean Ones", year: 1900, cast: [] }	Object
(8)	{ title: "Searching Ruins on Broadway, Galveston, for Dead Bodies", year: 1900, cast: [] }	Object
(9)	{ title: "The Tribulations of an Amateur Photographer", year: 1900, cast: [] }	Object
(10)	{ title: "Trouble in Hogan's Alley", year: 1900, cast: [] }	Object
(11)	{ title: "Boarding School Girls' Pajama Parade", year: 1900, cast: [] }	Object
(12)	{ title: "Two Old Sparks", year: 1900, cast: [] }	Object

8. Actualizar TODOS los documentos (películas) que tengan el array cast vacío, añadiendo un nuevo elemento dentro del array con valor Undefined. Cuidado! El tipo de cast debe seguir siendo un array. El array debe ser así -> ["Undefined"].

db.movies.updateMany(

```

{
  cast: { $exists: true, $type: "array", $size: 0 }
},
{
  $set: { cast: ["Undefined"] }
}

```

);

```

1 db.movies.updateMany(
2   {
3     cast: { $exists: true, $type: "array", $size: 0 }
4   },
5   {
6     $set: { cast: ["Undefined"] }
7   }
8 );

```

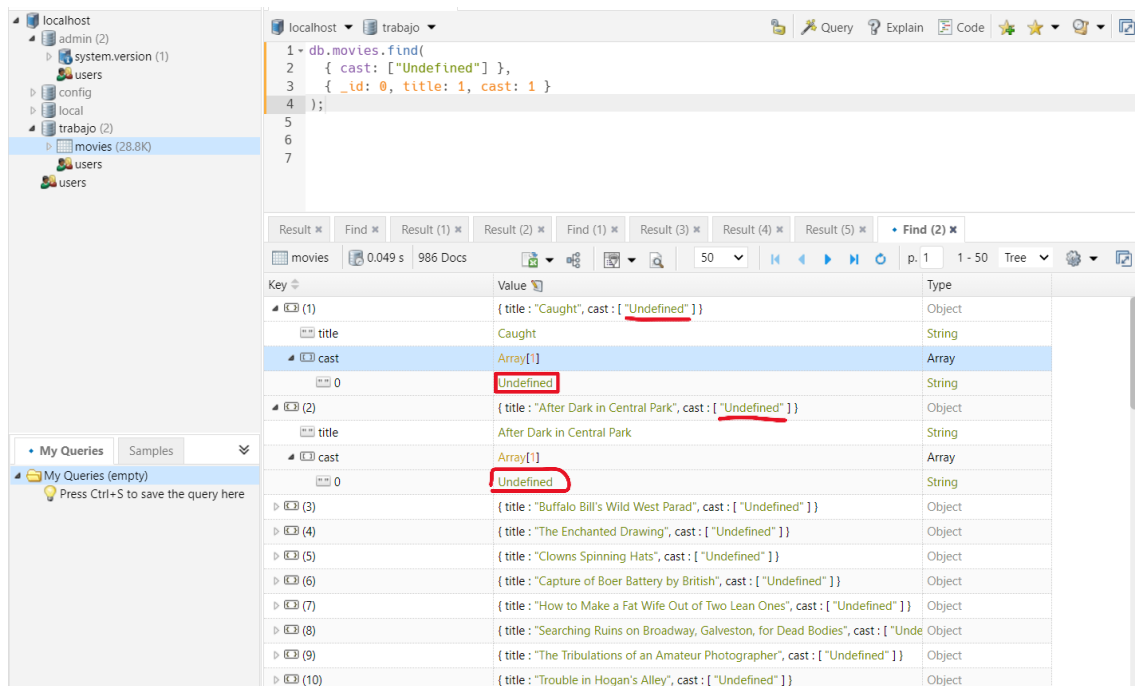
Key	Value	Type
(1)	{ acknowledged: true, matchedCount: 986, modifiedCount: 986 }	Object
acknowledged	true	Bool
matchedCount	986	Int32
modifiedCount	986	Int32

Nos aseguramos de que los 986 documentos se han actualizado correctamente. Comprobamos que 'cast' se haya actualizado con "Undefined":

```
db.movies.countDocuments({ cast: ["Undefined"] });
```



```
db.movies.find(  
  { cast: ["Undefined"] },  
  { _id: 0, title: 1, cast: 1 }  
);
```



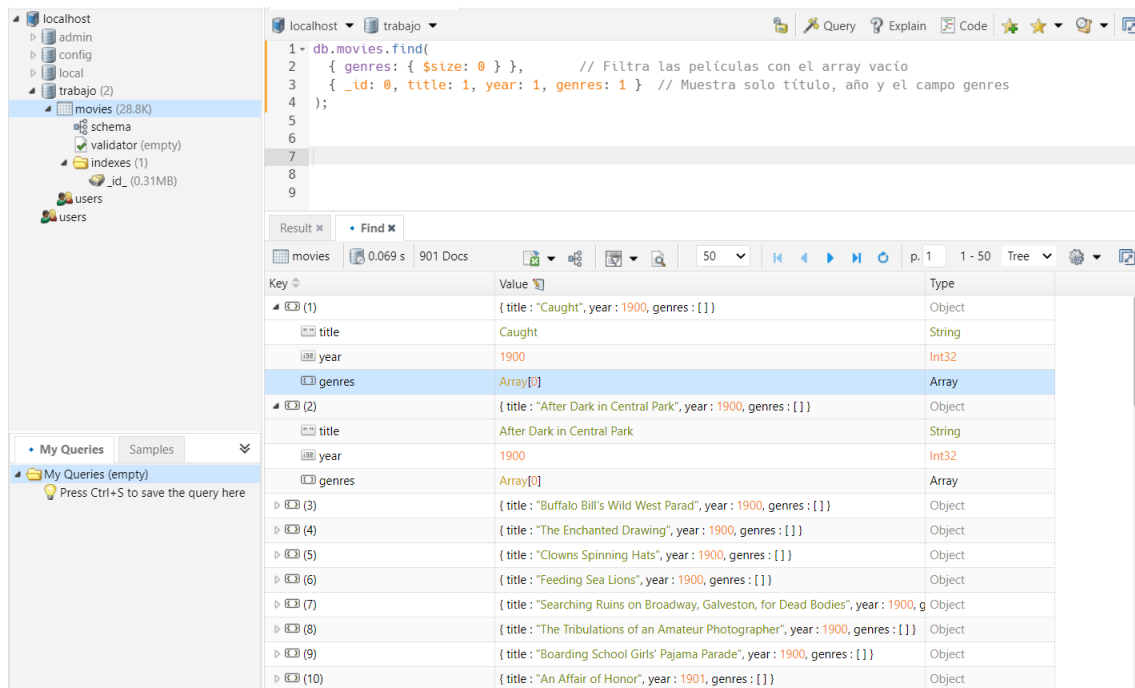
9. Contar cuantos documentos (películas) tienen el array genres vacío.

```
db.movies.countDocuments({ genres: { $size: 0 } });
```



Un total de 901 documentos tienen el array 'genres' vacío. Observamos como se visualiza este 'genres' vacío:

```
db.movies.find(
  { genres: { $size: 0 } },
  { _id: 0, title: 1, year: 1, genres: 1 }
);
```



The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including 'movies' (28.8K) and 'users'. The main panel displays a query result for the 'movies' collection. The query is: `db.movies.find({ genres: { $size: 0 } }, { _id: 0, title: 1, year: 1, genres: 1 })`. The result shows 901 documents. The first document is expanded, showing its structure: `{ title: 'Caught', year: 1900, genres: [] }`. The 'genres' field is highlighted, showing it is an empty array. Below the first document, a list of 10 documents is shown, all with empty 'genres' arrays.

Key	Value	Type
(1)	{ title: "Caught", year: 1900, genres: [] }	Object
title	Caught	String
year	1900	Int32
genres	Array[]	Array
(2)	{ title: "After Dark in Central Park", year: 1900, genres: [] }	Object
title	After Dark in Central Park	String
year	1900	Int32
genres	Array[]	Array
(3)	{ title: "Buffalo Bill's Wild West Parad", year: 1900, genres: [] }	Object
(4)	{ title: "The Enchanted Drawing", year: 1900, genres: [] }	Object
(5)	{ title: "Clowns Spinning Hats", year: 1900, genres: [] }	Object
(6)	{ title: "Feeding Sea Lions", year: 1900, genres: [] }	Object
(7)	{ title: "Searching Ruins on Broadway, Galveston, for Dead Bodies", year: 1900, genres: [] }	Object
(8)	{ title: "The Tribulations of an Amateur Photographer", year: 1900, genres: [] }	Object
(9)	{ title: "Boarding School Girls' Pajama Parade", year: 1900, genres: [] }	Object
(10)	{ title: "An Affair of Honor", year: 1901, genres: [] }	Object

10. Actualizar TODOS los documentos (películas) que tengan el array genres vacío, añadiendo un nuevo elemento dentro del array con valor Undefined. Cuidado! El tipo de genres debe seguir siendo un array. El array debe ser así -> ["Undefined"].

```
db.movies.updateMany(
  { genres: { $size: 0 } }, // Filtra solo los documentos con el array vacío
  { $set: { genres: ["Undefined"] } } // Reemplaza el array vacío con ["Undefined"]
);
```

```

1- db.movies.updateMany(
2-   { genres: { $size: 0 } }, // Filtra solo los documentos con el array vacío
3-   { $set: { genres: ["Undefined"] } } // Reemplaza el array vacío con ["Undefined"]
4- );

```

Key	Value	Type
(1)	{ acknowledged: true, matchedCount: 901, modifiedCount: 901 }	Object
acknowledged	true	Bool
matchedCount	901	Int32
modifiedCount	901	Int32

Comprobamos cómo se ve 'genres' tras este update:

```

db.movies.find(
  { genres: ["Undefined"] }, // Películas cuyo array 'genres' contiene exactamente "Undefined"
  { _id: 0, title: 1, year: 1, genres: 1 }
);

```

```

1- db.movies.find(
2-   { genres: ["Undefined"] }, // Busca las películas cuyo array 'genres' contiene exactamente "Undefined"
3-   { _id: 0, title: 1, year: 1, genres: 1 }
4- );

```

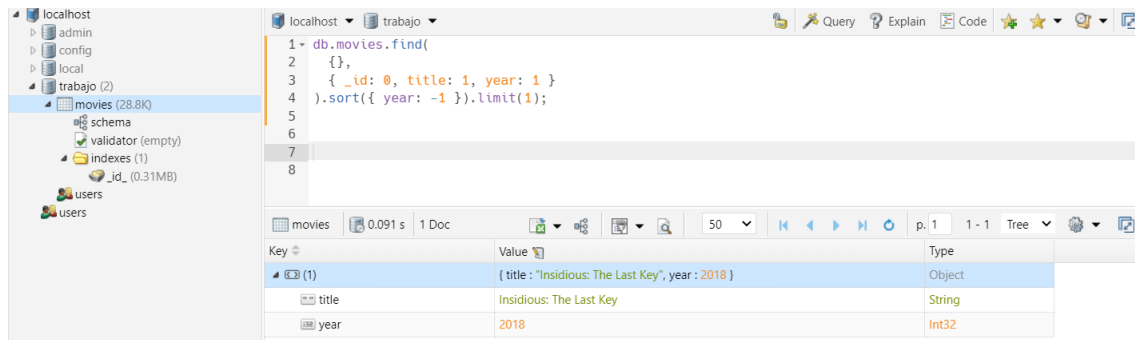
Key	Value	Type
(1)	{ title: "Caught", year: 1900, genres: ["Undefined"] }	Object
title	Caught	String
year	1900	Int32
genres	Array[1]	Array
0	Undefined	String
(2)	{ title: "After Dark in Central Park", year: 1900, genres: ["Undefined"] }	Object
(3)	{ title: "Buffalo Bill's Wild West Parad", year: 1900, genres: ["Undefined"] }	Object
(4)	{ title: "The Enchanted Drawing", year: 1900, genres: ["Undefined"] }	Object
(5)	{ title: "Clowns Spinning Hats", year: 1900, genres: ["Undefined"] }	Object
(6)	{ title: "Feeding Sea Lions", year: 1900, genres: ["Undefined"] }	Object
(7)	{ title: "Searching Ruins on Broadway, Galveston, for Dead Bodies", year: 1900, genres: ["Undefined"] }	Object
(8)	{ title: "The Tribulations of an Amateur Photographer", year: 1900, genres: ["Undefined"] }	Object
(9)	{ title: "Boarding School Girls' Pajama Parade", year: 1900, genres: ["Undefined"] }	Object
(10)	{ title: "An Affair of Honor", year: 1901, genres: ["Undefined"] }	Object
(11)	{ title: "Another Job for the Undertaker", year: 1901, genres: ["Undefined"] }	Object
(12)	{ title: "New Life Rescue", year: 1900, genres: ["Undefined"] }	Object

11. Mostrar el año más reciente / actual que tenemos sobre todas las películas.

```

db.movies.find(
  {},
  { _id: 0, title: 1, year: 1 }
).sort({ year: -1 }).limit(1);

```



2018 es el año más reciente/actual de las películas que contiene la base de datos.

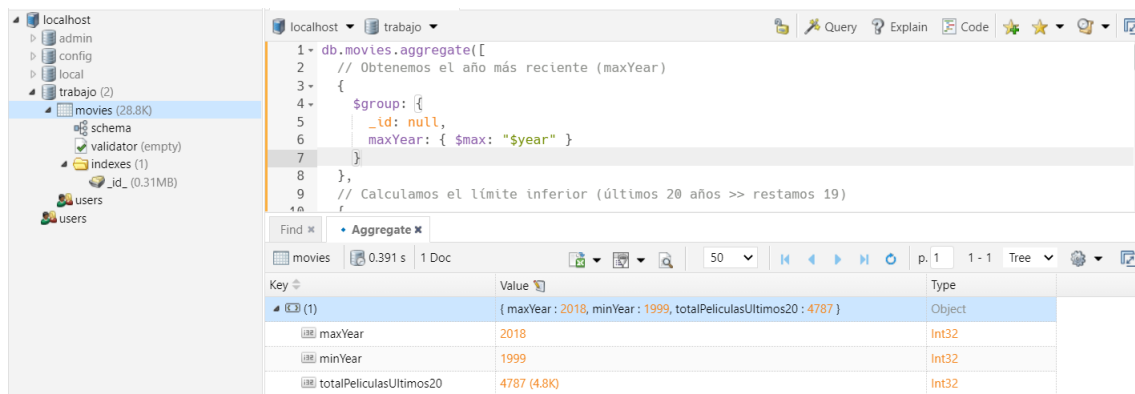
12. Contar cuántas películas han salido en los últimos 20 años. Debe hacerse desde el último año que se tienen registradas películas en la colección, no desde la fecha actual, mostrando el resultado total de esos años, no el resultado por año. Revisar que no se cuentan 21 años. Se debe hacer con el Framework de Agregación.

```
db.movies.aggregate([
  // Obtenemos el año más reciente (maxYear)
  {
    $group: {
      _id: null,
      maxYear: { $max: "$year" }
    }
  },
  // Calculamos el límite inferior (últimos 20 años >> restamos 19)
  {
    $project: {
      _id: 0,
      maxYear: 1,
      minYear: { $subtract: ["$maxYear", 19] }
    }
  },
  // Contamos las películas dentro del rango [minYear, maxYear]
  {
    $lookup: {
      from: "movies",
      let: { maxY: "$maxYear", minY: "$minYear" },
```

```

pipeline: [
  {
    $match: {
      $expr: {
        $and: [
          { $gte: ["$year", "$$minY"] },
          { $lte: ["$year", "$$maxY"] }
        ]
      }
    }
  },
  { $count: "totalPeliculasUltimos20" }
],
as: "conteo"
}
},
// Resultado final
{
  $project: {
    maxYear: 1,
    minYear: 1,
    totalPeliculasUltimos20: {
      $arrayElemAt: ["$conteo.totalPeliculasUltimos20", 0]
    }
  }
}
];

```



En los últimos 20 años desde el último registro de la colección, hay un total de 4787 películas. Si quisiésemos mostrar las películas de los últimos 20 años en lugar de contarlas, sustituiríamos '\$lookup' por una etapa de '\$unwind' + una etapa de '\$match':

```
db.movies.aggregate([
  // Obtenemos el año más reciente (maxYear)
  {
    $group: {
      _id: null,
      maxYear: { $max: "$year" }
    }
  },
  // Calculamos el límite inferior (últimos 20 años >> restamos 19)
  {
    $project: {
      _id: 0,
      maxYear: 1,
      minYear: { $subtract: ["$maxYear", 19] }
    }
  },
  // Unimos (join) la colección de películas consigo misma (self-join) para poder aplicar un filtro ($match)
  // que use las variables calculadas.
  {
    $lookup: {
      from: "movies", // La misma colección
      pipeline: [
        { $match: { _id: { $exists: true } } } // Devuelve todos los documentos de 'movies'
```

```

    ],
    as: "películas"
  }
},
// Desenrollamos el array de películas, creando un documento por cada película
{
  $unwind: "$películas"
},
// Filtramos las películas que cumplen con el rango de [minYear, maxYear]
{
  $match: {
    $expr: {
      $and: [
        { $gte: ["$películas.year", "$minYear"] }, // year >= minYear
        { $lte: ["$películas.year", "$maxYear"] } // year <= maxYear
      ]
    }
  }
},
// Proyectamos el documento final de la película, eliminando las variables de rango
{
  $replaceRoot: {
    newRoot: "$películas"
  }
}
]);

```



```

1 db.movies.aggregate([
2   // Obtenemos el año más reciente (maxYear)
3   {
4     $group: {
5       _id: null,
6       maxYear: { $max: "$year" }
7     }
8   },
9   // Calculamos el límite inferior (últimos 20 años >> restamos 19)
10  {
11    $sort: { maxYear: -1 }
12  },
13  { $limit: 19 }
14 ])

```

Key	Value	Type
(1) 6915ee7f9328d07e554c0e07	{ title: "10 Things I Hate About You", year: 1999 } (5 fields)	Document
_id (asc index)	6915ee7f9328d07e554c0e07	ObjectId
title	10 Things I Hate About You	String
year	1999	Int32
cast	Array[]	Array
genres	Array[]	Array
(2) 6915ee7f9328d07e554c0e08	{ title: "The 13th Warrior", year: 1999 } (5 fields)	Document
(3) 6915ee7f9328d07e554c0e09	{ title: "200 Cigarettes", year: 1999 } (5 fields)	Document
(4) 6915ee7f9328d07e554c0e0a	{ title: "The 4th Floor", year: 1999 } (5 fields)	Document
(5) 6915ee7f9328d07e554c0e0b	{ title: "8mm", year: 1999 } (5 fields)	Document
(6) 6915ee7f9328d07e554c0e0c	{ title: "American Movie", year: 1999, cast: ["Mark Borchardt", "Tom Schimmels"], genres: ["Drama"] } (7 fields)	Document
(7) 6915ee7f9328d07e554c0e0d	{ title: "American Beauty", year: 1999 } (5 fields)	Document
(8) 6915ee7f9328d07e554c0e0e	{ title: "American Pie", year: 1999 } (5 fields)	Document
(9) 6915ee7f9328d07e554c0e0f	{ title: "Angel's Dance", year: 1999 } (5 fields)	Document
(10) 6915ee7f9328d07e554c0e10	{ title: "Analyze This", year: 1999 } (5 fields)	Document
(11) 6915ee7f9328d07e554c0e11	{ title: "Anna and the King", year: 1999, cast: ["Jodie Foster", "Chow Yun-fat"], genres: ["Drama"] } (7 fields)	Document
(12) 6915ee7f9328d07e554c0e12	{ title: "Animal Farm", year: 1999 } (5 fields)	Document
(13) 6915ee7f9328d07e554c0e13	{ title: "The Adventures of Elmo in Grouchland", year: 1999 } (5 fields)	Document

```

1 db.movies.aggregate([
2   // Obtenemos el año más reciente (maxYear)
3   {
4     $group: {
5       _id: null,
6       maxYear: { $max: "$year" }
7     }
8   },
9   // Calculamos el límite inferior (últimos 20 años >> restamos 19)
10  {
11    $sort: { maxYear: -1 }
12  },
13  { $limit: 19 }
14 ])

```

Key	Value	Type
(4770) 6915ee7f9328d07e554c20ab	{ title: "The Silence", year: 2018 } (5 fields)	Document
(4771) 6915ee7f9328d07e554c20ac	{ title: "If Beale Street Could Talk", year: 2018 } (5 fields)	Document
(4772) 6915ee7f9328d07e554c20ad	{ title: "This One's for the Ladies", year: 2018 } (5 fields)	Document
(4773) 6915ee7f9328d07e554c20ae	{ title: "Ben Is Back", year: 2018 } (5 fields)	Document
(4774) 6915ee7f9328d07e554c20af	{ title: "Mortal Engines", year: 2018 } (5 fields)	Document
(4775) 6915ee7f9328d07e554c20b0	{ title: "The Mule", year: 2018 } (5 fields)	Document
(4776) 6915ee7f9328d07e554c20b1	{ title: "Spider-Man: Into the Spider-Verse", year: 2018 } (5 fields)	Document
(4777) 6915ee7f9328d07e554c20b2	{ title: "Vice", year: 2018 } (5 fields)	Document
(4778) 6915ee7f9328d07e554c20b3	{ title: "Widows", year: 2018 } (5 fields)	Document
(4779) 6915ee7f9328d07e554c20b4	{ title: "Second Act", year: 2018 } (5 fields)	Document
(4780) 6915ee7f9328d07e554c20b5	{ title: "Bumblebee", year: 2018 } (5 fields)	Document
(4781) 6915ee7f9328d07e554c20b6	{ title: "Mary Poppins Returns", year: 2018 } (5 fields)	Document
(4782) 6915ee7f9328d07e554c20b7	{ title: "Welcome to Marwen", year: 2018 } (5 fields)	Document
(4783) 6915ee7f9328d07e554c20b8	{ title: "Conundrum: Secrets Among Friends", year: 2018 } (5 fields)	Document
(4784) 6915ee7f9328d07e554c20b9	{ title: "Holmes and Watson", year: 2018 } (5 fields)	Document
(4785) 6915ee7f9328d07e554c20ba	{ title: "On the Basis of Sex", year: 2018 } (5 fields)	Document
(4786) 6915ee7f9328d07e554c20bb	{ title: "Aquaman", year: 2018 } (5 fields)	Document
(4787) 6915ee7f9328d07e554c20bc	{ title: "Destroyer", year: 2018 } (5 fields)	Document

Las capturas corresponden a los primeros y últimos documentos que pertenecen a esta colección. Las primeras películas que entrarían en esta colección serían del año “1999”, 20 años atrás del año del último registro de película en la base de datos, “2018”, como bien hemos obtenido en el ejercicio anterior. Las últimas películas de la colección son de “2018”, año más reciente en los documentos (películas) como bien hemos mencionado anteriormente.

**13. Contar cuántas películas han salido en la década de los 60 (del 60 al 69 incluidos).
Se debe hacer con el Framework de Agregación.**

```
db.movies.aggregate([
  // Filtramos solo las películas entre 1960 y 1969
  {
    $match: {
      year: { $gte: 1960, $lte: 1969 }
    }
  },
  // Agrupamos todo en un único grupo y sumamos 1 por cada película
  {
    $group: {
      _id: null,
      totalPelículasDecada60: { $sum: 1 }
    }
  },
  // Proyectamos el resultado final sin el _id
  {
    $project: {
      _id: 0,
      totalPelículasDecada60: 1
    }
  }
]);
```

```

1 db.movies.aggregate([
2   // Filtramos solo las películas entre 1960 y 1969
3   {
4     $match: {
5       year: { $gte: 1960, $lte: 1969 }
6     },
7   },
8   // Agrupamos todo en un único grupo y sumamos 1 por cada película
9   {
10    $group: {
11      _id: null,
12      totalPelículasDecada60: { $sum: 1 }
13    },
14  },
15  // Proyectamos el resultado final sin el _id
16  {
17    $project: {
18      _id: 0,
19      totalPelículasDecada60: 1
20    }
21  }
22 ]);
23

```

Key	Value	Type
totalPelículasDecada60	1414 (1.4K)	Int32

En la década de los 60, hay un total de 1414 películas. Visualizamos algunos registros de películas de estas fechas:

```

db.movies.aggregate([
  {
    $match: {
      year: { $gte: 1960, $lte: 1969 }
    }
  },
  {
    $project: {
      _id: 0,
      title: 1,
      year: 1
    }
  },
  {
    $sort: { year: 1, title: 1 } // opcional: ordena por año y título
  }
]);

```

```

1 db.movies.aggregate([
2   {
3     $match: {
4       year: { $gte: 1960, $lte: 1969 }
5     },
6   },
7   {
8     $project: {
9       _id: 0,
10      title: 1,
11      year: 1
12    },
13  },
14  {
15    $sort: { year: 1, title: 1 } // opcional: ordena por año y título
16  }
17 ]]);

```

Key	Value	Type
(1)	{ title: "12 to the Moon", year: 1960 }	Object
title	12 to the Moon	String
year	1960	Int32
(2)	{ title: "13 Fighting Men", year: 1960 }	Object
(3)	{ title: "13 Ghosts", year: 1960 }	Object
(4)	{ title: "5 Branded Women", year: 1960 }	Object

```

1 db.movies.aggregate([
2   {
3     $match: {
4       year: { $gte: 1960, $lte: 1969 }
5     },
6   },
7   {
8     $project: {
9       _id: 0,
10      title: 1,
11      year: 1
12    },
13  },
14  {
15    $sort: { year: 1, title: 1 } // opcional: ordena por año y título
16  }
17 ]]);

```

Key	Value	Type
(1406)	{ title: "They Shoot Horses, Don't They?", year: 1969 }	Object
(1407)	{ title: "Topaz", year: 1969 }	Object
(1408)	{ title: "Trilogy", year: 1969 }	Object
(1409)	{ title: "True Grit", year: 1969 }	Object
(1410)	{ title: "Viva Max!", year: 1969 }	Object
(1411)	{ title: "What Ever Happened to Aunt Alice?", year: 1969 }	Object
(1412)	{ title: "Where It's At", year: 1969 }	Object
(1413)	{ title: "Winning", year: 1969 }	Object
(1414)	{ title: "Young Billy Young", year: 1969 }	Object
title	Young Billy Young	String
year	1969	Int32

14. Mostrar el año u años con más películas mostrando el número de películas de ese año. Revisar si varios años pueden compartir tener el mayor número de películas.

```

db.movies.aggregate([

// Agrupamos por año y contamos cuántas películas hay en cada uno

{

  $group: {

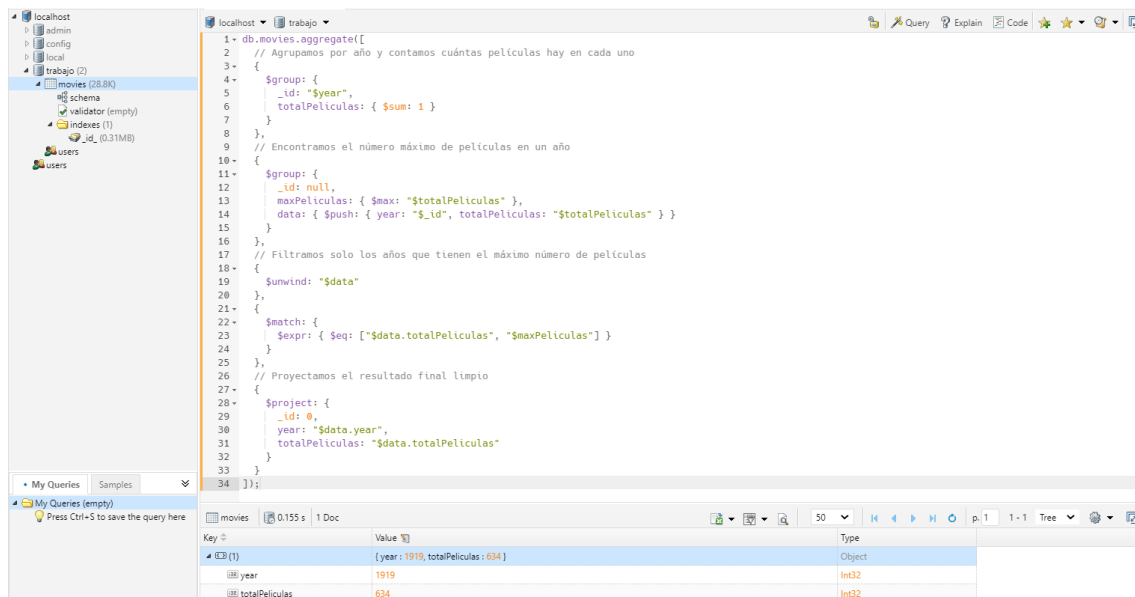
    _id: "$year",

```

```

        totalPeliculas: { $sum: 1 }
    }
},
// Encontramos el máximo número de películas en un año
{
    $group: {
        _id: null,
        maxPeliculas: { $max: "$totalPeliculas" },
        data: { $push: { year: "$_id", totalPeliculas: "$totalPeliculas" } }
    }
},
// Filtramos solo los años que tienen el máximo número de películas
{
    $unwind: "$data"
},
{
    $match: {
        $expr: { $eq: ["$data.totalPeliculas", "$maxPeliculas"] }
    }
},
// Proyectamos el resultado final limpio
{
    $project: {
        _id: 0,
        year: "$data.year",
        totalPeliculas: "$data.totalPeliculas"
    }
}
});

```



El año con más películas es, en este caso, un solo año “1919” con 634 películas.

15. Mostrar el año u años con menos películas mostrando el número de películas de ese año. Revisar si varios años pueden compartir tener el menor número de películas.

```
db.movies.aggregate([
```

```
// Agrupamos por año y contamos cuántas películas hay en cada uno
```

```
{
  $group: {
    _id: "$year",
    totalPelículas: { $sum: 1 }
  }
},
```

```
// Calculamos el número mínimo de películas por año
```

```
{
  $group: {
    _id: null,
    minPelículas: { $min: "$totalPelículas" },
    data: { $push: { year: "$_id", totalPelículas: "$totalPelículas" } }
  }
},
```

```
// Descomponemos el array 'data' para poder filtrar
```

```
{
```

```

    $unwind: "$data"

  },

  // Filtramos solo los años que tienen el número mínimo de películas

  {

    $match: {

      $expr: { $eq: ["$data.totalPelículas", "$minPelículas"] }

    }

  },

  // Mostramos el resultado final limpio

  {

    $project: {

      _id: 0,

      year: "$data.year",

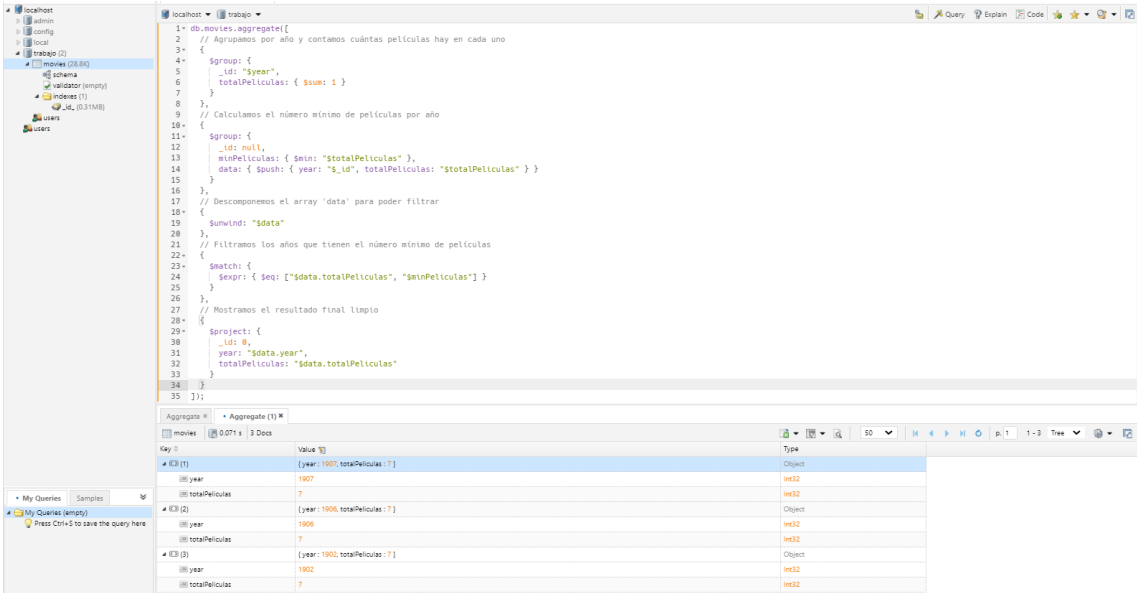
      totalPelículas: "$data.totalPelículas"

    }

  }

});

```



The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including a 'movies' collection. The main area displays a query in the 'Query' tab, which is a MongoDB aggregation pipeline. The query is as follows:

```

1- db.movies.aggregate([
2  // Agrupamos por año y contamos cuántas películas hay en cada uno
3  {
4    $group: {
5      _id: "$year",
6      totalPelículas: { $sum: 1 }
7    }
8  },
9  // Calculamos el número mínimo de películas por año
10 {
11   $group: {
12     _id: null,
13     minPelículas: { $min: "$totalPelículas" },
14     data: { $push: { year: "$_id", totalPelículas: "$totalPelículas" } }
15   }
16 },
17 // Descomponemos el array 'data' para poder filtrar
18 {
19   $unwind: "$data"
20 },
21 // Filtramos los años que tienen el número mínimo de películas
22 {
23   $match: {
24     $expr: { $eq: ["$data.totalPelículas", "$minPelículas"] }
25   }
26 },
27 // Mostramos el resultado final limpio
28 {
29   $project: {
30     _id: 0,
31     year: "$data.year",
32     totalPelículas: "$data.totalPelículas"
33   }
34 }
35 ]);

```

Below the query, the 'Results' tab shows the output of the aggregation. It displays three documents, each representing a year with the minimum number of movies (7):

Key	Value	Type
# (1) 1	{year: 1907, totalPelículas: 7}	Object
# (2) 2	{year: 1906, totalPelículas: 7}	Object
# (3) 3	{year: 1902, totalPelículas: 7}	Object

Los años con menos películas, en este caso, sí que son tres años empatados a menor registro de películas (1907, 1906, 1902), con un total de 7 películas cada uno.

16. Guardar en nueva colección llamada “actors” realizando la fase \$unwind por cast, no se debe agrupar de nuevo y debe contener todas las claves (title,year,cast,genres). Después, contar cuantos documentos existen en la nueva colección.

```
db.movies.aggregate([
  { $unwind: "$cast" },      // Descompone el array 'cast' -> un documento por actor
  {
    $project: {              // Conserva solo las claves solicitadas
      _id: 0,
      title: 1,
      year: 1,
      cast: 1,
      genres: 1
    }
  },
  { $out: "actors" }        // Guarda el resultado en 'actors'
]);
```



```
db.actors.countDocuments()
```



En la nueva colección llamada ‘actors’ hay 83224 documentos.

17. Sobre actors (nueva colección), mostrar la lista con los 5 actores que han participado en más películas mostrando el número de películas en las que ha participado. Importante! Se necesita previamente filtrar para descartar aquellos actores llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan.

```
db.actors.aggregate([
```



```

{
  $match: { cast: { $ne: "Undefined" } } // Filtramos actores "Undefined"
},
{
  $group: { // Agrupamos por nombre de actor
    _id: "$cast",
    moviesCount: { $sum: 1 } // Contamos cuántas películas tiene cada actor
  }
},
{
  $sort: { moviesCount: -1 } // Ordenamos de mayor a menor
},
{
  $limit: 5 // Solo los 5 primeros
}
]);

```

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible with collections 'actors' (83.2K) and 'movies' (28.8K). The main editor displays the following query:

```

1 db.actors.aggregate([
2   {
3     $match: { cast: { $ne: "Undefined" } } // Filtramos actores "Undefined"
4   },
5   {
6     $group: { // Agrupamos por nombre de actor
7       _id: "$cast",
8       moviesCount: { $sum: 1 } // Contamos cuántas películas tiene cada actor
9     }
10  },
11  {
12    $sort: { moviesCount: -1 } // Ordenamos de mayor a menor
13  },
14  {
15    $limit: 5 // Solo los 5 primeros
16  }
17 ]]);
18

```

Below the query editor, the results are displayed in a table format. The table has columns for 'Key', 'Value', and 'Type'. The results are as follows:

Key	Value	Type
(1) Harold Lloyd	{ moviesCount: 190 }	Document
_id (asc index)	Harold Lloyd	String
moviesCount	190	Int32
(2) Hoot Gibson	{ moviesCount: 142 }	Document
_id (asc index)	Hoot Gibson	String
moviesCount	142	Int32
(3) John Wayne	{ moviesCount: 136 }	Document
(4) Charles Starrett	{ moviesCount: 116 }	Document
(5) Bebe Daniels	{ moviesCount: 103 }	Document

Los 5 actores con más participaciones en películas son: Harold Lloyd (190), Hoot Gibson (142), John Wayne (136), Charles Starrett (116) y Bebe Daniels (103).

18. Sobre actores (nueva colección), agrupar por película y año mostrando las 5 en las que más actores hayan participado, mostrando el número total de actores.

```
db.actors.aggregate([
{
  $group: {
    _id: { title: "$title", year: "$year" },      // Agrupamos por película y año
    totalActors: { $sum: 1 }                     // Contamos un actor por documento
  },
{
  $sort: { totalActors: -1 }                     // Ordenamos de mayor a menor
},
{
  $limit: 5                                     // Tomamos solo las 5 primeras
}
]);
```

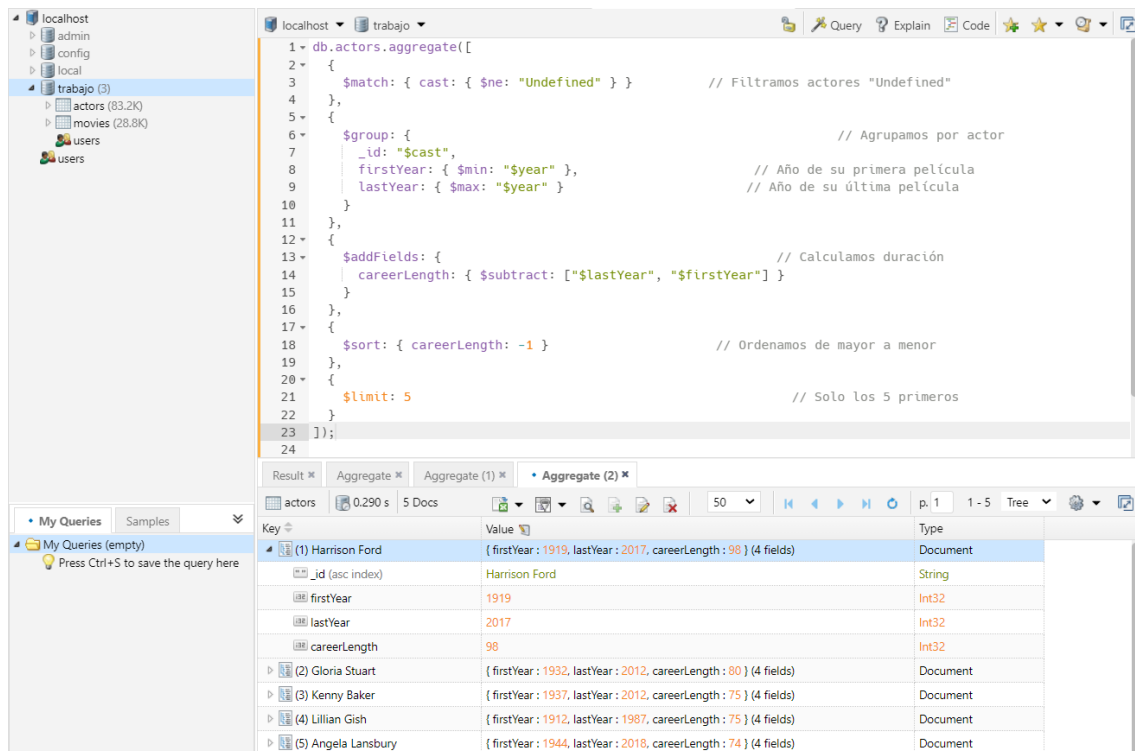
The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including the 'trabajo' database and the 'movies' collection. The main panel displays the aggregation query entered in the query editor. Below the query, the results are shown in a table format, displaying the top 5 movies by the number of actors.

Key	Value	Type
(1) { title: "The Twilight Saga: Breaking Down - Part 2", year: 2012 }	{ title: "The Twilight Saga: Breaking Down - Part 2", year: 2012 }	Document
_id (asc index)	{ title: "The Twilight Saga: Breaking Down - Part 2", year: 2012 }	Object
title	The Twilight Saga: Breaking Down - Part 2	String
year	2012	Int32
totalActors	35	Int32
(2) { title: "Anchorman 2: The Legend Continues", year: 2013 }	{ title: "Anchorman 2: The Legend Continues", year: 2013 }	Document
_id (asc index)	{ title: "Anchorman 2: The Legend Continues", year: 2013 }	Object
totalActors	33	Int32
(3) { title: "Cars 2", year: 2011 }	{ title: "Cars 2", year: 2011 }	Document
(4) { title: "Avengers: Infinity War", year: 2019 }	{ title: "Avengers: Infinity War", year: 2019 }	Document
(5) { title: "Grown Ups 2", year: 2015 }	{ title: "Grown Ups 2", year: 2015 }	Document

Las 5 películas con mayor nº de actores son: “The Twilight Saga: Breaking Down – Part 2” (35), “Anchorman 2: The Legend Continues” (33), “Cars 2” (32), “Avengers: Infinity War” (29) y “Grown Ups 2” (28).

19. Sobre actors (nueva colección), mostrar los 5 actores cuya carrera haya sido la más larga. Para ello, se debe mostrar cuándo comenzó su carrera, cuándo finalizó y cuántos años ha trabajado. Importante! Se necesita previamente filtrar para descartar aquellos actores llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan.

```
db.actors.aggregate([
  {
    $match: { cast: { $ne: "Undefined" } } // Filtramos actores "Undefined"
  },
  {
    $group: { // Agrupamos por actor
      _id: "$cast",
      firstYear: { $min: "$year" }, // Año de su primera película
      lastYear: { $max: "$year" } // Año de su última película
    }
  },
  {
    $addFields: { // Calculamos duración
      careerLength: { $subtract: ["$lastYear", "$firstYear"] }
    }
  },
  {
    $sort: { careerLength: -1 } // Ordenamos de mayor a menor
  },
  {
    $limit: 5 // Solo los 5 primeros
  }
]);
```



```

1 db.actors.aggregate([
2   {
3     $match: { cast: { $ne: "Undefined" } } // Filtramos actores "Undefined"
4   },
5   {
6     $group: { // Agrupamos por actor
7       _id: "$cast",
8       firstYear: { $min: "$year" }, // Año de su primera película
9       lastYear: { $max: "$year" } // Año de su última película
10    },
11  },
12  {
13    $addFields: { // Calculamos duración
14      careerLength: { $subtract: ["$lastYear", "$firstYear"] }
15    },
16  },
17  {
18    $sort: { careerLength: -1 } // Ordenamos de mayor a menor
19  },
20  {
21    $limit: 5 // Solo los 5 primeros
22  }
23 ]);
24

```

Key	Value	Type
(1) Harrison Ford	{ firstYear: 1919, lastYear: 2017, careerLength: 98 } (4 fields)	Document
_id (asc index)	Harrison Ford	String
firstYear	1919	Int32
lastYear	2017	Int32
careerLength	98	Int32
(2) Gloria Stuart	{ firstYear: 1932, lastYear: 2012, careerLength: 80 } (4 fields)	Document
(3) Kenny Baker	{ firstYear: 1937, lastYear: 2012, careerLength: 75 } (4 fields)	Document
(4) Lillian Gish	{ firstYear: 1912, lastYear: 1987, careerLength: 75 } (4 fields)	Document
(5) Angela Lansbury	{ firstYear: 1944, lastYear: 2018, careerLength: 74 } (4 fields)	Document

Los 5 actores cuyas carreras son más longevas son: Harrison Ford (98), Gloria Stuart (80), Kenny Baker (75), Lillian Gish (75) y Angela Lansbury (74).

20. Sobre actors (nueva colección), Guardar en nueva colección llamada “genres” realizando la fase \$unwind por genres, no se debe agrupar de nuevo y debe contener todas las claves (title,year,cast,genres). Después, contar cuantos documentos existen en la nueva colección.

```

db.actors.aggregate([

{

  $unwind: "$genres" // Descomponemos el array 'genres' -> un documento por género

},

{

  $project: { // Conservamos todas las claves solicitadas

    _id: 0,

    title: 1,

    year: 1,

    cast: 1,

    genres: 1

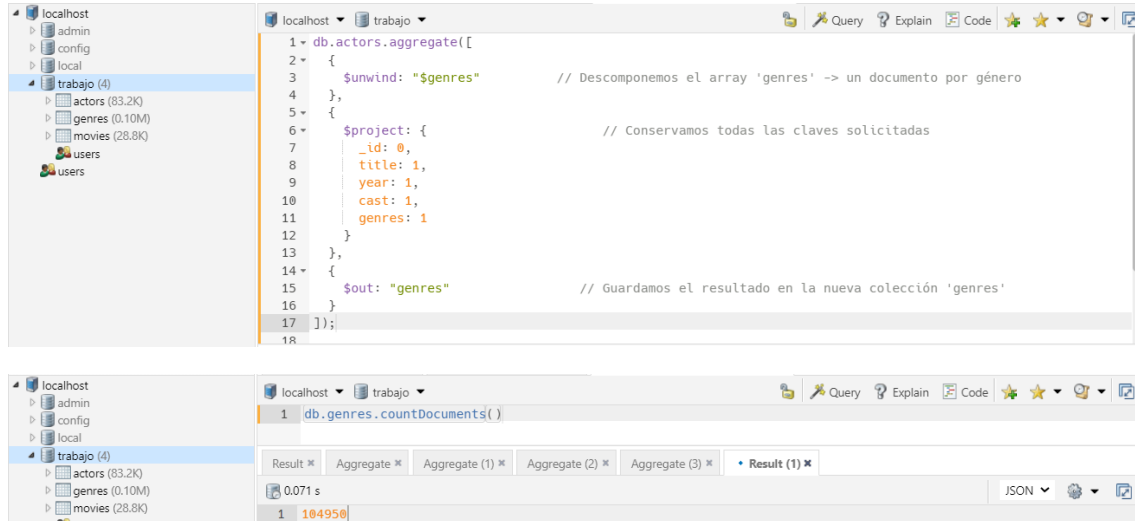
  }

},

]

```

```
{
  $out: "genres" // Guardamos el resultado en la nueva colección 'genres'
}
});
```



En la nueva colección llamada ‘genres’ hay 104950 documentos.

21. Sobre genres (nueva colección), mostrar los 5 documentos agrupados por “Año y Género” que más número de películas diferentes tienen mostrando el número total de películas. Importante! Se necesita previamente filtrar para descartar aquellos genres llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan.

```
db.genres.aggregate([
{
  $match: { genres: { $ne: "Undefined" } } // Filtramos géneros "Undefined"
},
{
  $group: { // Agrupamos por año y género
    _id: { year: "$year", genres: "$genres" },
    totalMovies: { $addToSet: "$title" } // Guardamos los títulos únicos
  }
},
{
  $project: {
    _id: 1,
```

```

    totalMovies: { $size: "$totalMovies" }           // Contamos el número de películas diferentes
  }
},
{
  $sort: { totalMovies: -1 }                         // Ordenamos de mayor a menor
},
{
  $limit: 5                                          // Solo los 5 primeros
}
]);

```

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including collections like 'actores', 'genres', 'movies', and 'users'. The main panel displays a query in the 'Query' tab, which is a MongoDB aggregation pipeline. The query is as follows:

```

1 - db.genres.aggregate([
2 - {
3 -   $match: { genres: { $ne: "Undefined" } }      // Filtramos géneros "Undefined"
4 - },
5 - {
6 -   $group: {                                     // Agrupamos por año y género
7 -     _id: { year: "$year", genres: "$genres" },
8 -     totalMovies: { $addToSet: "$title" }        // Guardamos los títulos únicos
9 -   },
10 - },
11 - {
12 -   $project: {
13 -     _id: 1,
14 -     totalMovies: { $size: "$totalMovies" }     // Contamos el número de películas diferentes
15 -   },
16 - },
17 - {
18 -   $sort: { totalMovies: -1 }                   // Ordenamos de mayor a menor
19 - },
20 - {
21 -   $limit: 5                                    // Solo los 5 primeros
22 - },
23 - ]);

```

Below the query, the 'Results' tab shows the output of the aggregation. It displays a table with 5 documents, each representing a genre and its total number of movies. The first document is for the year 1919, genre 'Drama', with a total of 291 movies. The other documents show results for the years 1925, 1924, 1919, and 1922, all for the 'Drama' genre.

Key	Value	Type
(1) { year: 1919, genres: "Drama" }	{ totalMovies: 291 }	Document
year	1919	Int32
genres	Drama	String
totalMovies	291	Int32
(2) { year: 1925, genres: "Drama" }	{ totalMovies: 247 }	Document
(3) { year: 1924, genres: "Drama" }	{ totalMovies: 233 }	Document
(4) { year: 1919, genres: "Comedy" }	{ totalMovies: 226 }	Document
(5) { year: 1922, genres: "Drama" }	{ totalMovies: 209 }	Document

22. Sobre genres (nueva colección), mostrar los 5 actores y los géneros en los que han participado con más número de géneros diferentes, se debe mostrar tanto el número de géneros diferentes que ha interpretado como los diferentes géneros además del nombre del actor. Importante! Se necesita previamente filtrar para descartar aquellos actores llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan.

```

db.genres.aggregate([
{
  $match: { cast: { $ne: "Undefined" } }           // Filtramos actores "Undefined"
}

```

```

    },
    {
      $group: { // Agrupamos por actor
        _id: "$cast",
        uniqueGenres: { $addToSet: "$genres" } // Obtenemos géneros distintos
      }
    },
    {
      $addFields: { // Contamos cuántos géneros distintos tiene
        numGenres: { $size: "$uniqueGenres" }
      }
    },
    {
      $sort: { numGenres: -1 } // Ordenamos de mayor a menor
    },
    {
      $limit: 5 // Tomamos los 5 primeros
    }
  ];
}
);

```

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including collections like 'actors', 'genres', 'movies', and 'users'. The main panel displays a query in the 'Query' tab, which is a MongoDB aggregation pipeline. The query is as follows:

```

1 db.genres.aggregate([
2   {
3     $match: { cast: { $ne: "Undefined" } } // Filtramos actores "Undefined"
4   },
5   {
6     $group: { // Agrupamos por actor
7       _id: "$cast",
8       uniqueGenres: { $addToSet: "$genres" } // Obtenemos géneros distintos
9     },
10  },
11  {
12    $addFields: { // Contamos cuántos géneros distintos tiene
13      numGenres: { $size: "$uniqueGenres" }
14    },
15  },
16  {
17    $sort: { numGenres: -1 } // Ordenamos de mayor a menor
18  },
19  {
20    $limit: 5 // Tomamos los 5 primeros
21  }
22 ]);
23

```

Below the query, the 'Results' tab shows the output of the aggregation. It displays a table with 5 documents, each representing an actor and their unique genres.

Key	Value	Type
(1) Dennis Quaid	{ numGenres : 20 } (3 fields)	Document
(2) James Mason	{ numGenres : 19 } (3 fields)	Document
(3) Michael Caine	{ numGenres : 19 } (3 fields)	Document
(4) Colin Farrell	{ numGenres : 18 } (3 fields)	Document
(5) Lionel Barrymore	{ numGenres : 18 } (3 fields)	Document

Los 5 actores que han participado en un mayor nº de géneros son: Dennis Quaid (20), James Mason (19), Michael Caine (19), Colin Farrell (18) y Lionel Barrymore (18).

A continuación, mostramos todos los géneros en los que ha participado Dennis Quaid para ver cómo se muestra:

Key	Value	Type
(1) Dennis Quaid	{ numGenres : 20 } (3 fields)	Document
Dennis Quaid	Dennis Quaid	String
uniqueGenres	Array[20]	Array
0	Animated	String
1	Thriller	String
2	Disaster	String
3	Biography	String
4	Fantasy	String
5	Musical	String
6	Suspense	String
7	Crime	String
8	Romance	String
9	Dance	String
10	Science Fiction	String
11	Family	String
12	Drama	String
13	Western	String
14	Action	String
15	Comedy	String
16	Sports	String
17	Adventure	String
18	Satire	String
19	Horror	String
numGenres	20	Int32
(2) James Mason	{ numGenres : 19 } (3 fields)	Document
(3) Michael Caine	{ numGenres : 19 } (3 fields)	Document
(4) Colin Farrell	{ numGenres : 18 } (3 fields)	Document
(5) Lionel Barrymore	{ numGenres : 18 } (3 fields)	Document

23. Sobre genres (nueva colección), mostrar las 5 películas y su año correspondiente en los que más géneros diferentes han sido catalogados, mostrando esos géneros y el número de géneros que contiene. Importante! Se necesita previamente filtrar para descartar aquellos genres llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan. Se muestra una salida de ejemplo, el resultado es erróneo.

```
db.genres.aggregate([
  {
    $match: { genres: { $ne: "Undefined" } }    // Filtramos géneros "Undefined"
  },
  {
    $group: {
      _id: { title: "$title", year: "$year" },    // Agrupamos por película (título y año)
      uniqueGenres: { $addToSet: "$genres" }    // Guardamos los géneros distintos
    }
  },
  {
    $addFields: {
```



```

numGenres: { $size: "$uniqueGenres" } // Contamos el número de géneros distintos

}

},

{

  $sort: { numGenres: -1 } // Ordenamos de mayor a menor

},

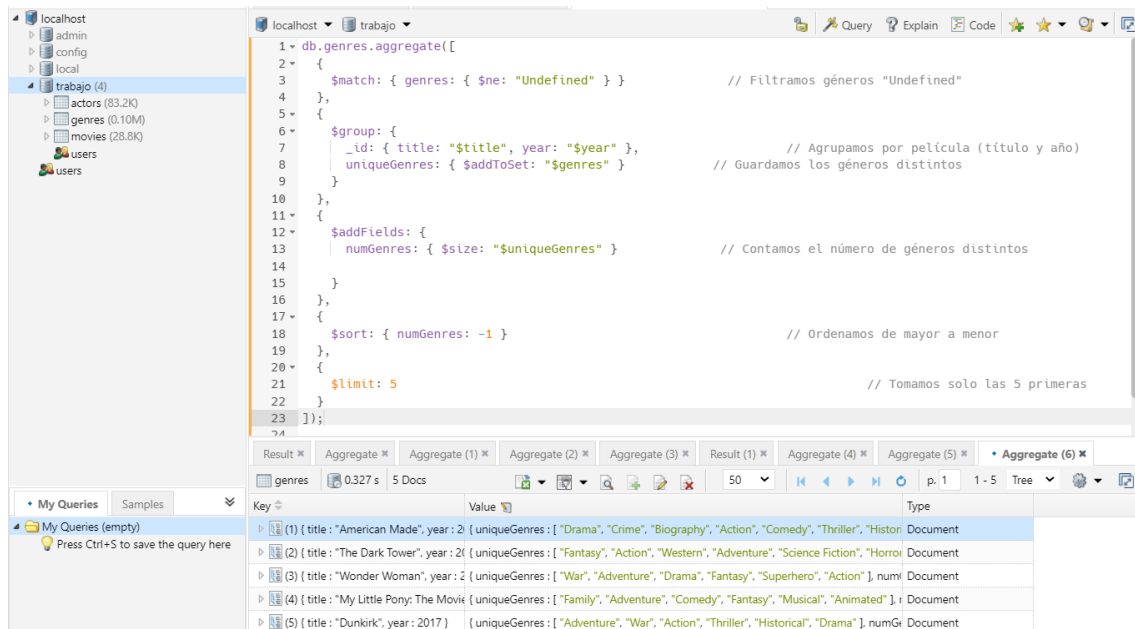
{

  $limit: 5 // Tomamos solo las 5 primeras

}

});

```



The screenshot shows a MongoDB query editor interface. The query is as follows:

```

1 db.genres.aggregate([
2   {
3     $match: { genres: { $ne: "Undefined" } } // Filtramos géneros "Undefined"
4   },
5   {
6     $group: {
7       _id: { title: "$title", year: "$year" }, // Agrupamos por película (título y año)
8       uniqueGenres: { $addToSet: "$genres" } // Guardamos los géneros distintos
9     }
10  },
11  {
12    $addFields: {
13      numGenres: { $size: "$uniqueGenres" } // Contamos el número de géneros distintos
14    }
15  },
16  {
17    $sort: { numGenres: -1 } // Ordenamos de mayor a menor
18  },
19  {
20    $limit: 5 // Tomamos solo las 5 primeras
21  }
22 ]);
23

```

The results tab shows 5 documents:

Key	Value	Type
(1)	{ title: "American Made", year: 2017, uniqueGenres: ["Drama", "Crime", "Biography", "Action", "Comedy", "Thriller", "Historical"], numGenres: 7 }	Document
(2)	{ title: "The Dark Tower", year: 2017, uniqueGenres: ["Fantasy", "Action", "Western", "Adventure", "Science Fiction", "Horror"], numGenres: 6 }	Document
(3)	{ title: "Wonder Woman", year: 2016, uniqueGenres: ["War", "Adventure", "Drama", "Fantasy", "Superhero", "Action"], numGenres: 6 }	Document
(4)	{ title: "My Little Pony: The Movie", year: 2017, uniqueGenres: ["Family", "Adventure", "Comedy", "Fantasy", "Musical", "Animated"], numGenres: 6 }	Document
(5)	{ title: "Dunkirk", year: 2017, uniqueGenres: ["Adventure", "War", "Action", "Thriller", "Historical", "Drama"], numGenres: 6 }	Document

Mostramos cómo se ven estos géneros en las películas:

localhost

admin

config

local

trabajo (4)

actors (83.2K)

genres (0.10M)

movies (28.8K)

users

My Queries

Samples

My Queries (empty)

Press Ctrl+S to save the query here

localhost ▾ trabajo ▾

1 db.genres.aggregate([

Result ▾ Aggregate ▾ Aggregate (1) ▾ Aggregate (2) ▾ Aggregate (3) ▾ Result (1) ▾ Aggregate (4) ▾ Aggregate (5) ▾ Aggregate (6) ▾

genres 0.327 s 5 Docs

Key	Value	Type
1 { title: "American Made", year: 2017 }	{ uniqueGenres: ["Drama", "Crime", "Biography", "Action", "Comedy", "Thriller", "Historical"], numGenres: 7 }	Document
▸ _id (asc index)	{ title: "American Made", year: 2017 }	Object
▸ uniqueGenres	Array[7]	Array
▸ 0	Drama	String
▸ 1	Crime	String
▸ 2	Biography	String
▸ 3	Action	String
▸ 4	Comedy	String
▸ 5	Thriller	String
▸ 6	Historical	String
▸ numGenres	7	Int32
2 { title: "The Dark Tower", year: 2017 }	{ uniqueGenres: ["Fantasy", "Action", "Western", "Adventure", "Science Fiction", "Horror"], numGenres: 6 }	Document
▸ _id (asc index)	{ title: "The Dark Tower", year: 2017 }	Object
▸ uniqueGenres	Array[6]	Array
▸ 0	Fantasy	String
▸ 1	Action	String
▸ 2	Western	String
▸ 3	Adventure	String
▸ 4	Science Fiction	String
▸ 5	Horror	String
▸ numGenres	6	Int32
3 { title: "Wonder Woman", year: 2017 }	{ uniqueGenres: ["War", "Adventure", "Drama", "Fantasy", "Superhero", "Action"], numGenres: 6 }	Document
▸ _id (asc index)	{ title: "Wonder Woman", year: 2017 }	Object
▸ uniqueGenres	Array[6]	Array
▸ numGenres	6	Int32
4 { title: "My Little Pony: The Movie", year: 2017 }	{ uniqueGenres: ["Family", "Adventure", "Comedy", "Fantasy", "Musical", "Animated"], numGenres: 6 }	Document
▸ _id (asc index)	{ title: "My Little Pony: The Movie", year: 2017 }	Object
▸ uniqueGenres	Array[6]	Array
▸ numGenres	6	Int32
5 { title: "Dunkirk", year: 2017 }	{ uniqueGenres: ["Adventure", "War", "Action", "Thriller", "Historical", "Drama"], numGenres: 6 }	Document
▸ _id (asc index)	{ title: "Dunkirk", year: 2017 }	Object
▸ uniqueGenres	Array[6]	Array
▸ numGenres	6	Int32

Las 5 películas con mayor nº de géneros son: “American Made” (7), “The Dark Tower” (6), “Wonder Woman” (6), “My Little Pony: The Movie” (6) y “Dunkirk” (6).

Querys Libres

Lo primero de todo, especificar que todas las querys libres partirán con la colección “movies” como base, no desde ‘actors’ o desde ‘genres’.

Query Libre 1: Géneros más populares

En esta query, mostraremos los géneros que más aparecen en las películas:

```
db.movies.aggregate([
  // Descomponemos el array 'genres'
  { $unwind: "$genres" },
  // Filtramos valores no informativos
  { $match: {
    "genres": {
      $nin: ["Undefined", "", null]
    }
  } },
  { $group: {
    _id: "$genres"
  } },
  { $sort: {
    _id: -1
  } },
  { $limit: 5 }
])
```

// Agrupamos por el nombre del género y contamos

```
{ $group: {  
  _id: "$genres",  
  num_peliculas: { $sum: 1 } // Contamos las apariciones del género  
}}
```

// Ordenamos por conteo de forma descendente

```
{ $sort: { num_peliculas: -1 } }
```

});

The screenshot shows the MongoDB Compass interface. On the left, the database structure is visible, including collections like 'actors', 'genres', 'schema', 'validator', 'indexes', 'movies', and 'users'. The 'movies' collection is selected. The main area displays a query in the 'Query' tab:

```
1 db.movies.aggregate([  
2 // Expandimos el array 'genres' para que cada documento contenga solo un género, permitiendo el conteo individual  
3 { $unwind: "$genres" },  
4  
5 // Filtramos para asegurar la limpieza de los datos, excluyendo valores no informativos como "Undefined" o vacíos  
6 { $match: {  
7   "genres": {  
8     $nin: ["Undefined", "", null]  
9   }  
10 } },  
11  
12 // Agrupamos por el nombre del género y utilizamos $sum como acumulador para contar el número total de películas  
13 { $group: {  
14   _id: "$genres", // La clave de agrupación es el nombre del género  
15   num_peliculas: { $sum: 1 } // Acumulador (cuenta cuántas veces aparece ese género)  
16 } },  
17  
18 // Ordenamos toda la lista de resultados por 'num_peliculas' de forma descendente  
19 { $sort: { num_peliculas: -1 } }  
20 ]])
```

Below the query, the 'Aggregate (3)' results are shown. The table has columns 'Key', 'Value', and 'Type'. The results are as follows:

Key	Value	Type
(1) Drama	{ num_peliculas : 8744 }	Document
_id (asc index)	Drama	String
num_peliculas	8744 (8.7K)	Int32
(2) Comedy	{ num_peliculas : 7363 }	Document
(3) Western	{ num_peliculas : 3013 }	Document
(4) Crime	{ num_peliculas : 1499 }	Document
(5) Horror	{ num_peliculas : 1166 }	Document
(6) Musical	{ num_peliculas : 1166 }	Document
(7) Romance	{ num_peliculas : 1147 }	Document
(8) Action	{ num_peliculas : 1109 }	Document
(9) Adventure	{ num_peliculas : 999 }	Document

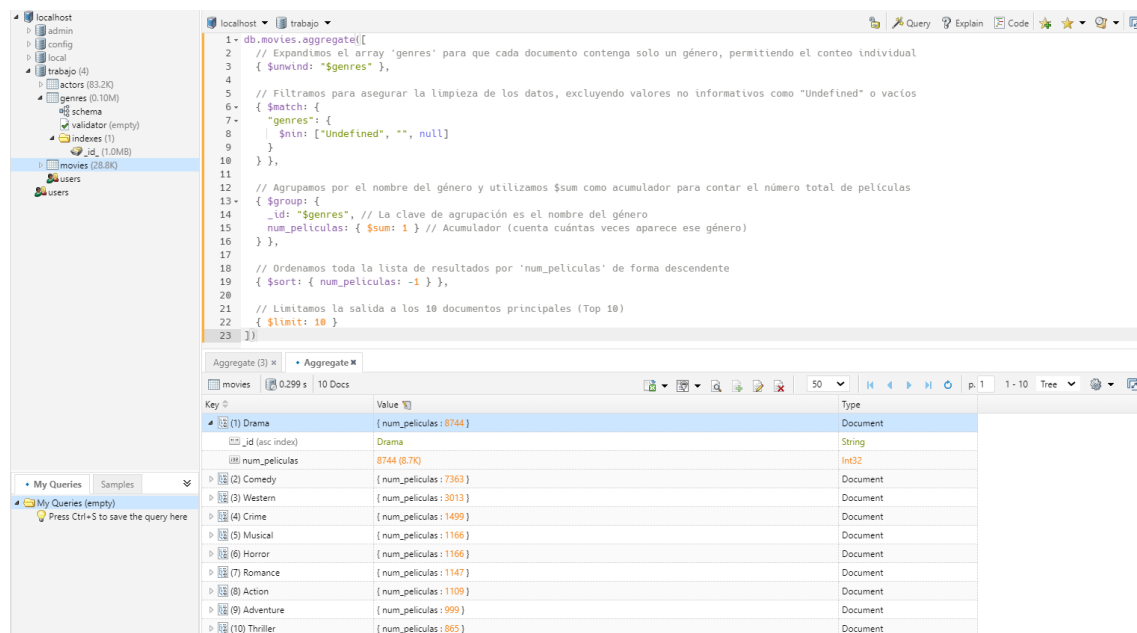
The screenshot shows the MongoDB Compass interface with the same query as above. The 'Aggregate (3)' results are shown in a scrollable table. The table has columns 'Key', 'Value', and 'Type'. The results are as follows:

Key	Value	Type
(1) Drama	{ num_peliculas : 8744 }	Document
_id (asc index)	Drama	String
num_peliculas	8744 (8.7K)	Int32
(2) Comedy	{ num_peliculas : 7363 }	Document
(3) Western	{ num_peliculas : 3013 }	Document
(4) Crime	{ num_peliculas : 1499 }	Document
(5) Horror	{ num_peliculas : 1166 }	Document
(6) Musical	{ num_peliculas : 1166 }	Document
(7) Romance	{ num_peliculas : 1147 }	Document
(8) Action	{ num_peliculas : 1109 }	Document
(9) Adventure	{ num_peliculas : 999 }	Document
(10) Thriller	{ num_peliculas : 865 }	Document
(11) Science Fiction	{ num_peliculas : 780 }	Document
(12) Animated	{ num_peliculas : 757 }	Document
(13) Mystery	{ num_peliculas : 629 }	Document
(14) War	{ num_peliculas : 556 }	Document
(15) Documentary	{ num_peliculas : 529 }	Document
(16) Biography	{ num_peliculas : 439 }	Document
(17) Noir	{ num_peliculas : 403 }	Document
(18) Family	{ num_peliculas : 381 }	Document
(19) Short	{ num_peliculas : 355 }	Document
(20) Fantasy	{ num_peliculas : 327 }	Document
(21) Sports	{ num_peliculas : 139 }	Document
(22) Suspense	{ num_peliculas : 111 }	Document
(23) Historical	{ num_peliculas : 106 }	Document
(24) Superheroes	{ num_peliculas : 83 }	Document
(25) Spy	{ num_peliculas : 71 }	Document
(26) Satire	{ num_peliculas : 58 }	Document
(27) Erotic	{ num_peliculas : 36 }	Document
(28) Disaster	{ num_peliculas : 32 }	Document
(29) Performance	{ num_peliculas : 22 }	Document
(30) Teen	{ num_peliculas : 21 }	Document
(31) Martial Arts	{ num_peliculas : 21 }	Document
(32) Slasher	{ num_peliculas : 15 }	Document
(33) Dance	{ num_peliculas : 14 }	Document
(34) Political	{ num_peliculas : 14 }	Document
(35) Supernatural	{ num_peliculas : 11 }	Document
(36) Live Action	{ num_peliculas : 9 }	Document
(37) Sport	{ num_peliculas : 8 }	Document
(38) Silent	{ num_peliculas : 7 }	Document
(39) Independent	{ num_peliculas : 2 }	Document
(40) Found Footage	{ num_peliculas : 1 }	Document
(41) Legal	{ num_peliculas : 1 }	Document

El género más popular es “Drama” con 8744 películas, y los menos populares “Found Footage” y “Legal” con 1 película.

Elegí la query “Géneros más Populares” porque sirve como ejercicio de conteo de frecuencia para manejar arrays de manera eficiente: utilizo ‘\$unwind’ para descomponer el array de géneros y así poder contar cada elemento de forma individual; luego, aplico ‘\$match’ para realizar una limpieza de datos que elimina valores vacíos o "Undefined"; finalmente, ‘\$group’ realiza el conteo real usando ‘\$sum’ para sumar las apariciones, y ‘\$sort’ ordena la lista para que los géneros con más películas aparezcan al principio, demostrando cómo transformar datos anidados en un ranking de popularidad.

Decido coger todos los datos y no hacer un top, ya que puede ser relevante en el contexto de una empresa al mostrar todos los géneros, no solo los más populares, lo que permite detectar tendencias, nichos de mercado y oportunidades que podrían pasar desapercibidas si usara ‘\$limit’. Aun así, observamos como se vería un top 10 ({ \$limit: 10 }):



```
1 db.movies.aggregate([
2   // Expandimos el array 'genres' para que cada documento contenga solo un género, permitiendo el conteo individual
3   { $unwind: "$genres" },
4
5   // Filtramos para asegurar la limpieza de los datos, excluyendo valores no informativos como "Undefined" o vacíos
6   { $match: {
7     "genres": {
8       $nin: ["Undefined", "", null]
9     }
10  } },
11
12  // Agrupamos por el nombre del género y utilizamos $sum como acumulador para contar el número total de películas
13  { $group: {
14    _id: "$genres", // La clave de agrupación es el nombre del género
15    num_peliculas: { $sum: 1 } // Acumulador (cuenta cuántas veces aparece ese género)
16  } },
17
18  // Ordenamos toda la lista de resultados por 'num_peliculas' de forma descendente
19  { $sort: { num_peliculas: -1 } },
20
21  // Limitamos la salida a los 10 documentos principales (Top 10)
22  { $limit: 10 }
23 ])
```

Key	Value	Type
(1) Drama	{ num_peliculas : 8744 }	Document
_id (asc index)	Drama	String
num_peliculas	8744 (8.7K)	Int32
(2) Comedy	{ num_peliculas : 7363 }	Document
(3) Western	{ num_peliculas : 3013 }	Document
(4) Crime	{ num_peliculas : 1499 }	Document
(5) Musical	{ num_peliculas : 1166 }	Document
(6) Horror	{ num_peliculas : 1166 }	Document
(7) Romance	{ num_peliculas : 1147 }	Document
(8) Action	{ num_peliculas : 1109 }	Document
(9) Adventure	{ num_peliculas : 999 }	Document
(10) Thriller	{ num_peliculas : 865 }	Document

Query Libre 2: Actor con más películas por década

```
db.movies.aggregate([

  // Expandimos el array 'cast' para generar un documento por cada aparición de un actor

  { $unwind: "$cast" },

  // Filtramos los documentos para excluir actores con el valor "Undefined"

  { $match: { cast: { $ne: "Undefined" } } },

  // Agrupamos por la combinación (Década + Actor) y cuenta la frecuencia

  // La década se calcula con operadores matemáticos: año - (año % 10)

  { $group: {
```

```

    _id: {
      decada: { $subtract: [ "$year", { $mod: [ "$year", 10 ] } ] },
      actor: "$cast"
    },
    count_movies: { $sum: 1 }
  } },

  // Ordenamos los resultados por década y, luego, por conteo descendente (el actor más popular de cada
  // década se posiciona primero)

  { $sort: { "_id.decada": 1, "count_movies": -1 } },

  // Volvemos a agrupar solo por la década. Usamos $first para capturar el actor y el conteo del primer
  // documento de cada grupo

  { $group: {
    _id: "$_id.decada",
    actor_top: { $first: "$_id.actor" },
    max_movies: { $first: "$count_movies" }
  } },

  // Formateamos la salida para transformar la década numérica en un rango de texto (YYYY-YYYY)

  { $project: {
    _id: 0,
    Decada: {
      $concat: [
        { $toString: "$_id" },
        "-",
        { $toString: { $add: [ "$_id", 9 ] } }
      ]
    },
    Actor_top: "$actor_top",
    Num_Peliculas: "$max_movies",
  } },

  // Ordenamos el resultado final por la década cronológicamente

  { $sort: { "Decada": 1 } }

]);

```

```

1 db.movies.aggregate([
2   // Expandimos el array 'cast' para generar un documento por cada aparición de un actor
3   { $unwind: "$cast" },
4
5   // Filtramos los documentos para excluir actores con el valor "Undefined"
6   { $match: { "cast": { $ne: "Undefined" } } },
7
8   // Agrupamos por la combinación (Década + Actor) y cuenta la frecuencia
9   // La década se calcula con operadores matemáticos: año - (año % 10)
10  { $group: {
11    _id: {
12      decade: { $subtract: [ "$year", { $mod: [ "$year", 10 ] } ] },
13      actor: "$cast"
14    },
15    count_movies: { $sum: 1 }
16  } },
17
18  // Ordenamos los resultados por década y, luego, por conteo descendente (el actor más popular de cada década se posiciona primero)
19  { $sort: { "_id.decade": 1, "count_movies": -1 } },
20
21  // Volvemos a agrupar solo por la década. Usamos $first para capturar el actor y el conteo del primer documento de cada grupo
22  { $group: {
23    _id: "$_id.decade",
24    actor_top: { $first: "$_id.actor" },
25    max_movies: { $first: "$count_movies" }
26  } },
27
28  // Formateamos la salida para transformar la década numérica en un rango de texto (YYYY-YYYY).
29  { $project: {
30    _id: 0,
31    Decada: {
32      $concat: [
33        { $toString: "$_id" },
34        "-",
35        { $toString: { $add: [ "$_id", 9 ] } }
36      ]
37    },
38    Actor_Top: "$actor_top",
39    Num_Peliculas: "$max_movies"
40  } },
41
42  // Ordenamos el resultado final por la década cronológicamente.
43  { $sort: { "Decada": 1 } }
44 ])

```

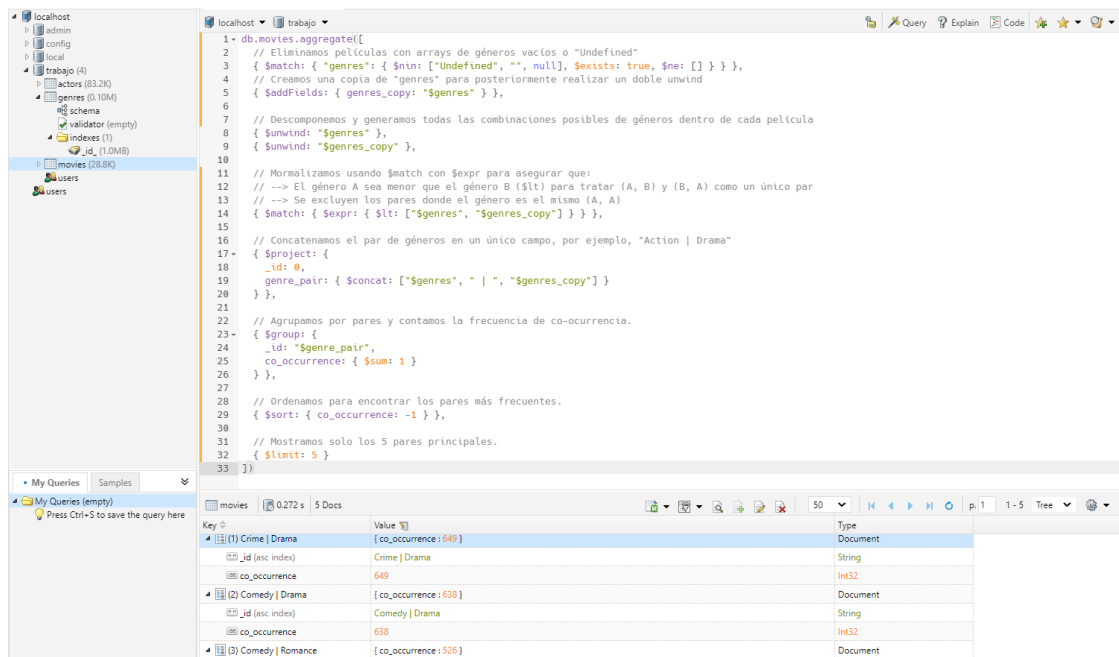
Key	Value	Type
(1)	{ Decada: "1900-1909", Actor_Top: "Florence Lawrence", Num_Peliculas: 7 }	Object
Decada	1900-1909	String
Actor_Top	Florence Lawrence	String
Num_Peliculas	7	Int32
(2)	{ Decada: "1910-1919", Actor_Top: "Harold Lloyd", Num_Peliculas: 160 }	Object
Decada	1910-1919	String
Actor_Top	Harold Lloyd	String
Num_Peliculas	160	Int32
(3)	{ Decada: "1920-1929", Actor_Top: "Hoot Gibson", Num_Peliculas: 99 }	Object
(4)	{ Decada: "1930-1939", Actor_Top: "John Wayne", Num_Peliculas: 54 }	Object
(5)	{ Decada: "1940-1949", Actor_Top: "Charles Starrett", Num_Peliculas: 60 }	Object
(6)	{ Decada: "1950-1959", Actor_Top: "Tom and Jerry", Num_Peliculas: 55 }	Object
(7)	{ Decada: "1960-1969", Actor_Top: "Dean Martin", Num_Peliculas: 26 }	Object
(8)	{ Decada: "1970-1979", Actor_Top: "Robert Duvall", Num_Peliculas: 20 }	Object
(9)	{ Decada: "1980-1989", Actor_Top: "Burt Reynolds", Num_Peliculas: 20 }	Object
(10)	{ Decada: "1990-1999", Actor_Top: "Bruce Willis", Num_Peliculas: 25 }	Object
(11)	{ Decada: "2000-2009", Actor_Top: "Samuel L. Jackson", Num_Peliculas: 22 }	Object
(12)	{ Decada: "2010-2019", Actor_Top: "Anna Kendrick", Num_Peliculas: 21 }	Object

Elegí la consulta “Actor con más películas por década” porque es un problema avanzado dentro del pipeline de agregación. La clave está en usar varias etapas juntas: hacer el cálculo de la década usando las funciones matemáticas (\$subtract y \$mod) directamente en el paso de \$group, lo que ahorra mucho código. Lo más importante es cómo usar la combinación de \$sort y el doble \$group con \$first. Esto obliga a MongoDB a ordenar los datos de forma que el actor más popular suba al primer puesto en cada década, y luego el segundo \$group lo captura, siendo una solución muy eficiente.

Query Libre 3: Pares de géneros más comunes

Esta query muestra las combinaciones de géneros que más aparecen juntas en películas, por ejemplo, comedia y romance.

```
db.movies.aggregate([
  // Eliminamos películas con arrays de géneros vacíos o "Undefined"
  { $match: { "genres": { $nin: ["Undefined", "", null], $exists: true, $ne: [] } } },
  // Creamos una copia de 'genres' para posteriormente realizar un doble unwind.
  { $addFields: { genres_copy: "$genres" } },
  // Descomponemos y generamos todas las combinaciones posibles de géneros dentro de cada película.
  { $unwind: "$genres" },
  { $unwind: "$genres_copy" },
  // Normalizamos usando $match con $expr para asegurar que:
  // >> El Género A sea menor que el Género B ($lt) para tratar (A, B) y (B, A) como un único par.
  // >> Se excluyan los pares donde el género es el mismo (A, A).
  { $match: { $expr: { $lt: ["$genres", "$genres_copy"] } } },
  // Concatenamos el par de géneros en un único campo (ej: "Action | Drama").
  { $project: {
    _id: 0,
    genre_pair: { $concat: ["$genres", " | ", "$genres_copy"] }
  } },
  // Agrupamos por pares y contamos la frecuencia de co-ocurrencia.
  { $group: {
    _id: "$genre_pair",
    co_occurrence: { $sum: 1 }
  } },
  // Ordenamos para encontrar los pares más frecuentes.
  { $sort: { co_occurrence: -1 } },
  // Mostramos solo los 5 pares principales.
  { $limit: 5 }
]);
```

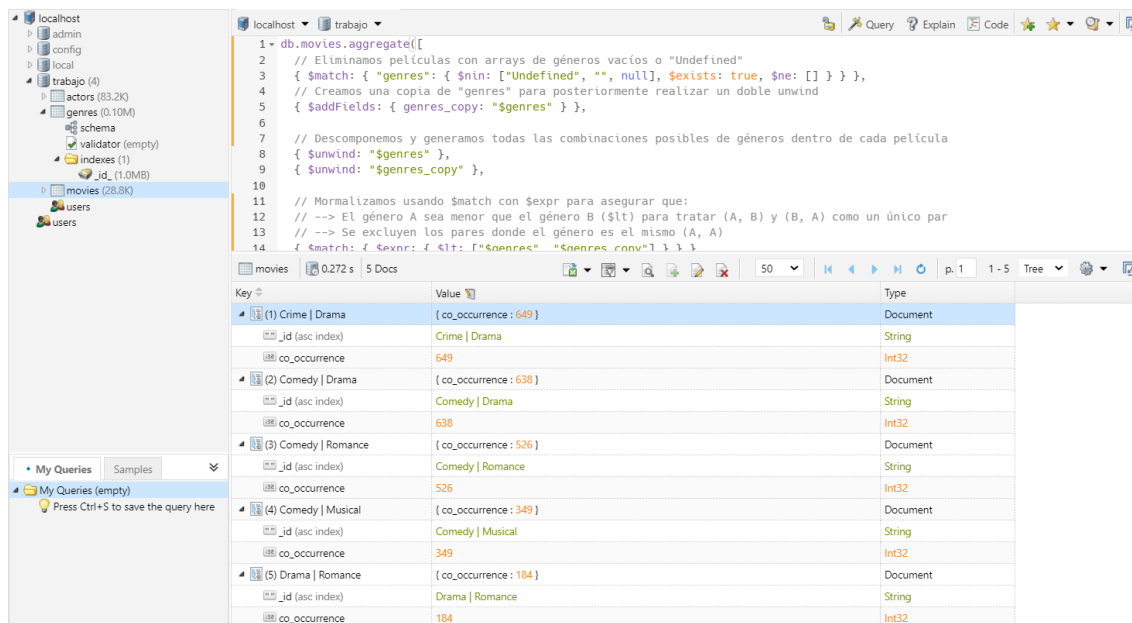


```

1 db.movies.aggregate([
2   // Eliminamos películas con arrays de géneros vacíos o "Undefined"
3   { $match: { "genres": { $nin: ["Undefined", "", null], $exists: true, $ne: [] } } },
4   // Creamos una copia de "genres" para posteriormente realizar un doble unwind
5   { $addFields: { genres_copy: "$genres" } },
6
7   // Descomponemos y generamos todas las combinaciones posibles de géneros dentro de cada película
8   { $unwind: "$genres" },
9   { $unwind: "$genres_copy" },
10
11   // Normalizamos usando $match con $expr para asegurar que:
12   // --> El género A sea menor que el género B ($lt) para tratar (A, B) y (B, A) como un único par
13   // --> Se excluyen los pares donde el género es el mismo (A, A)
14   { $match: { $expr: { $lt: [ "$genres", "$genres_copy" ] } } },
15
16   // Concatenamos el par de géneros en un único campo, por ejemplo, "Action | Drama"
17   { $project: {
18     _id: 0,
19     genre_pair: { $concat: [ "$genres", " | ", "$genres_copy" ] }
20   } },
21
22   // Agrupamos por pares y contamos la frecuencia de co-ocurrencia.
23   { $group: {
24     _id: "$genre_pair",
25     co_occurrence: { $sum: 1 }
26   } },
27
28   // Ordenamos para encontrar los pares más frecuentes.
29   { $sort: { co_occurrence: -1 } },
30
31   // Mostramos solo los 5 pares principales.
32   { $limit: 5 }
33 ])

```

Key	Value	Type
(1) Crime Drama	{ co_occurrence: 649 }	Document
_id (asc index)	Crime Drama	String
co_occurrence	649	Int32
(2) Comedy Drama	{ co_occurrence: 638 }	Document
_id (asc index)	Comedy Drama	String
co_occurrence	638	Int32
(3) Comedy Romance	{ co_occurrence: 526 }	Document



```

1 db.movies.aggregate([
2   // Eliminamos películas con arrays de géneros vacíos o "Undefined"
3   { $match: { "genres": { $nin: ["Undefined", "", null], $exists: true, $ne: [] } } },
4   // Creamos una copia de "genres" para posteriormente realizar un doble unwind
5   { $addFields: { genres_copy: "$genres" } },
6
7   // Descomponemos y generamos todas las combinaciones posibles de géneros dentro de cada película
8   { $unwind: "$genres" },
9   { $unwind: "$genres_copy" },
10
11   // Normalizamos usando $match con $expr para asegurar que:
12   // --> El género A sea menor que el género B ($lt) para tratar (A, B) y (B, A) como un único par
13   // --> Se excluyen los pares donde el género es el mismo (A, A)
14   { $match: { $expr: { $lt: [ "$genres", "$genres_copy" ] } } }
15 ])

```

Key	Value	Type
(1) Crime Drama	{ co_occurrence: 649 }	Document
_id (asc index)	Crime Drama	String
co_occurrence	649	Int32
(2) Comedy Drama	{ co_occurrence: 638 }	Document
_id (asc index)	Comedy Drama	String
co_occurrence	638	Int32
(3) Comedy Romance	{ co_occurrence: 526 }	Document
_id (asc index)	Comedy Romance	String
co_occurrence	526	Int32
(4) Comedy Musical	{ co_occurrence: 349 }	Document
_id (asc index)	Comedy Musical	String
co_occurrence	349	Int32
(5) Drama Romance	{ co_occurrence: 184 }	Document
_id (asc index)	Drama Romance	String
co_occurrence	184	Int32

Elegí la query “Top 5 Pares de géneros más comunes” porque analiza relaciones entre los datos. Para ver qué géneros van juntos, uso el doble ‘\$unwind’. Primero, descompongo el array de géneros en dos campos (‘genres’ y ‘genres_copy’) y los descompongo de nuevo, lo que crea todas las combinaciones posibles de géneros que hay en una película. La parte más difícil es usar la lógica condicional ‘\$expr’ con ‘\$lt’ en el ‘\$match’, clave para normalizar los pares: si tengo "Drama" y "Acción", evito que me cuente también "Acción" y "Drama". Al contar solo una vez cada par único, manejo arrays complejos y evito duplicados antes de usar el ‘\$group’ final para obtener el ranking de co-ocurrencia.